

Spectral Canvas

Field Guide

Everything the project plan actually specifies, the signal theory you need in order to build it, and a line-by-line reading of the encoder code that already exists in this repository — written for someone picking the project up cold.

REPOSITORY

Spectral-Canvas @ 2670f22

SOURCE DOCUMENT

Spectral Canvas Complete Project Plan, 20 pp.

HEADLINE RESULT

100 % pixel recovery measured from `black1.wav`

CODE REVIEWED

3 encoder files, 188 lines

STATE OF BUILD

Encoder complete, decoder and channel not started

GUIDE COVERS

Plan, theory, features, testing, code

This guide describes the code; it does not change it. Every number quoted was measured by running against the committed audio, with the repository left untouched.

Contents

1	The system in one page	3
1.1	The exact state the signal passes through	3
2	The plan, distilled	5
2.1	The core mapping	5
2.2	Inputs the system must accept	5
2.3	Preprocessing rules	5
2.4	The encoder mathematics	6
2.5	Output and synchronisation	6
2.6	The channel simulator, and what each effect looks like	7
2.7	The decoder	7
2.8	Metrics	8
2.9	Recommended starting parameters	8
3	The signal concepts, bit by bit	9
3.1	Sampling and the discrete-time axis	9
3.2	The Nyquist limit	9
3.3	Sinusoids and superposition	9
3.4	The DFT, the FFT, and bin width	10
3.5	The time–frequency resolution trade-off	10
3.6	Spectral leakage and windows	10
3.7	The STFT and the spectrogram	11
3.8	LTI systems, impulse response and convolution	11
3.9	Frequency response and filtering	12
3.10	Aliasing and resampling	12
3.11	Clipping, and why nonlinearity is different	12
3.12	Noise, SNR, and processing gain	13
3.13	Normalisation, headroom and crest factor	13
3.14	Thresholding and scoring	13
3.15	Cross-reference: where each concept lands in the code	14
4	Feature inventory	15
5	The six build phases, and how each is tested	17
5.1	Phase 1 — a reliable processed image matrix (complete)	17
5.2	Phase 2 — the image visibly appears in the spectrogram (complete)	17
5.3	Phase 3 — file-to-file recovery works (next up)	18
5.4	Phase 4 — controlled channel experiments work	18
5.5	Phase 5 — a polished software demonstration	19
5.6	Phase 6 — speaker to microphone (optional extension)	19
6	The existing code, line by line	20
6.1	encoder/image_preprocessor.py	20
6.2	encoder/audio_encoder.py	24
6.3	encoder/__pycache__/visualize.py	29
6.4	The two supporting files	30
7	Measurements taken from the real output	31
8	Open issues to raise with your teammate	33

PART 1

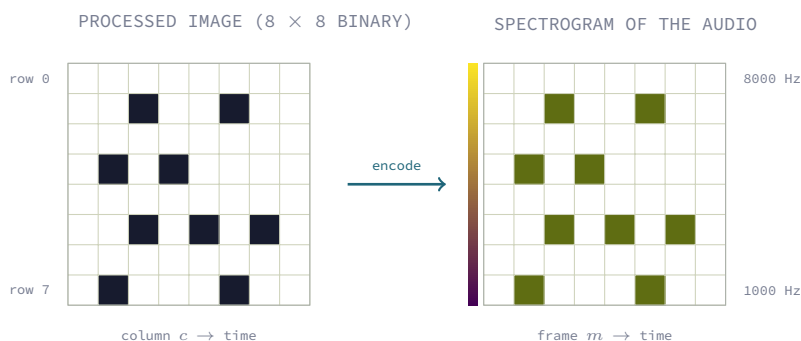
The system in one page

A **spectrogram** is a picture of sound: time runs left to right, frequency runs bottom to top, and loudness is brightness. An **image** is also a picture: columns run left to right, rows run bottom to top, and pixel value is brightness. Those two grids have the same shape. Spectral Canvas exploits that coincidence literally.

THE WHOLE IDEA

Give every **image row** its own **frequency**. Give every **image column** its own **slice of time**. Make a pixel's **brightness** the **loudness** of that tone during that slice. Play the result and the picture is inside the sound. Take its spectrogram and the picture comes back out.

That mapping is the entire project. Everything else — the windowing, the normalisation, the channel simulator, the STFT decoder — exists to make the round trip survive contact with a real signal chain.



Rows become frequencies, columns become time frames. The top row is the highest tone, so the picture is not upside down. In this repository the mapping is 64 rows across 1000–8000 Hz and 64 columns of 0.1 s each.

1.1 The exact state the signal passes through

Every stage below is implemented today except the last three. Follow this chain and you can point at any line of the existing code and say what it is holding.

NO.	STATE OF THE DATA	CONCRETE FORM RIGHT NOW	NEXT STATE VIA
1	Image file on disk	images/black1.png, RGB, about 9 MB	.convert("L")
2	Grayscale intensity field	2-D uint8, 0–255, full resolution	LANCZOS resize + pad
3	Low-resolution grayscale	64 × 64 uint8, white margins	threshold at 128
4	Binary pixel matrix	64 × 64 of {0, 1}; dark = 1 = tone on	additive synthesis
5	One frame of audio	4410 float64 samples, one chord	× Hann window
6	Tapered frame	same 4410 samples, faded in and out	np.concatenate
7	Full waveform	282 240 samples = 6.4 s	scale to 0.8 peak
8	Normalised waveform	peak exactly 0.8, RMS 0.057	wavfile.write
9	WAV file + key	mono float64 @ 44.1 kHz, metadata.json	TO DO channel
10	Received waveform	$y[n] = x[n] * h[n] + w[n]$	TO DO STFT decoder
11	Magnitude matrix	64 × 64 of $ X[c, k_r] $	normalise, threshold
12	Recovered image	64 × 64 of {0, 1}	—

READ THIS TWICE

Metrics compare state 12 against **state 4** — the processed 64 × 64 matrix — never against state 1, the original photograph. The plan says this explicitly, and it is the single most common way to make your own results look worse than they are.

PART 2

The plan, distilled

Everything below is specification lifted from the project plan: the parts that tell you what to build and how the mathematics has to work. Team allocation, week-by-week administration and grading criteria are omitted deliberately.

2.1 The core mapping

VISUAL PROPERTY	AUDIO / SPECTROGRAM PROPERTY
Image column	One audio time frame
Image row	One assigned audio frequency
Active pixel (binary)	Tone present
Inactive pixel	Tone absent
Grayscale intensity	Tone amplitude
Image width	Number of frames, hence total duration
Image height	Number of distinct encoding frequencies

2.2 Inputs the system must accept

- **Drawing canvas** — low resolution, drawn live during the demonstration.
- **Uploaded high-contrast images** — logos, icons, silhouettes, line art. The most reliable class, because the decoder only has to decide present or absent.
- **Uploaded grayscale photographs** — one large subject, plain background. Crowded scenes with fine detail do not survive.
- **Rendered text** — the user types a word and it is drawn into a high-contrast bitmap.
- **Generated QR codes** — optional, but the only input with a hard pass/fail outcome.

INPUT TYPE	SUGGESTED FIRST RESOLUTION	PREFERRED MODE
Drawing or symbol	64 × 32 or 64 × 64	Binary
Text or logo	64 × 32 or 96 × 48	Binary
QR code	32 × 32 or 64 × 64	Binary
Simple photograph	64 × 32 or 128 × 64	Grayscale
Detailed photograph	128 × 64, only later	Grayscale

2.3 Preprocessing rules

The original file cannot be encoded directly: its height would demand thousands of distinct frequencies and its width would demand minutes of audio. The preprocessor's job is to produce a matrix that fits the chosen

bandwidth and duration.

- Crop the region of interest; **resize preserving aspect ratio**; pad rather than stretch.
- RGB → grayscale; grayscale → binary at a selectable threshold.
- Brightness and contrast adjustment, and a black/white invert switch.
- Preserve the QR quiet zone when in QR mode.
- Show a preview of the *exact* matrix that will be transmitted, and store it separately from the original upload.

2.4 The encoder mathematics

Row r of an R -row image gets a frequency:

$$f_r = f_{\min} + r \frac{f_{\max} - f_{\min}}{R - 1}$$

In practice the orientation is reversed, so that row 0 (the top) gets $f_{\max}$ and the spectrogram is not upside down. The plan requires that orientation in words even though the formula as written ascends.

Amplitude comes straight from the pixel: $A[r, c] \in \{0, 1\}$ in binary mode, $A[r, c] = I[r, c]/255$ in grayscale mode. Each column becomes one frame:

$$x_c[n] = \sum_r A[r, c] \cos\left(2\pi f_r \frac{n}{f_s} + \varphi_r\right)$$

Frames are windowed, concatenated, and the whole waveform is normalised once:

$$x_{\text{win}}[n] = x[n] w[n], \quad x_{\text{out}}[n] = \frac{0.8 x[n]}{\max |x[n]|}$$

Duration is determined entirely by width: $N_{\text{frame}} = f_s T_f$ and $T_{\text{total}} = C T_f$.

IMAGE WIDTH	DURATION AT 20 MS/COL AT 100 MS/COL (THIS REPO)	
32 columns	0.64 s	3.2 s
64 columns	1.28 s	6.4 s
128 columns	2.56 s	12.8 s
256 columns	5.12 s	25.6 s

2.5 Output and synchronisation

WAV is the primary format — uncompressed, so no codec artefact confuses early debugging. Decoding a WAV you generated yourself needs no synchronisation, because sample 0 is the start of column 0. A *recording* does: it begins with unknown silence and room noise. The remedy is a short known preamble.

[sync tone or chirp] → [short silence] → [encoded image] → [optional end tone]

The decoder finds the start by energy threshold, by looking for the known frequency, or by correlating against a stored copy of the sync waveform. All three are far simpler than OFDM synchronisation, which the plan explicitly rules out.

2.6 The channel simulator, and what each effect looks like

This is the part that converts a clever visual trick into a Signals and Systems laboratory. Everything is one equation:

$$y[n] = x[n] * h[n] + w[n]$$

EFFECT	HOW IT IS APPLIED	WHAT YOU SEE IN THE RECOVERED IMAGE
White noise	Add Gaussian $w[n]$ at a chosen SNR	Random bright speckle; false pixels everywhere
Echo	$h[n] = \delta[n] + \alpha \delta[n - D]$	Columns copied to the right — horizontal smearing
Low-pass	Cut above f_c	The top rows vanish
High-pass	Cut below f_c	The bottom rows vanish
Band-stop	Cut a range	A horizontal band is erased from the middle
Narrow notch	Cut one frequency	One single row disappears
Resampling	Downsample with / without anti-alias	High rows <i>fold</i> down and appear mirrored
Clipping	Hard-limit the amplitude	Harmonics paint ghost rows that were never drawn

WHY THIS IS THE GOOD PART

“A low-pass filter reduces detail” is a sentence. “A low-pass filter at 4 kHz deletes rows 0 through 36 of the picture” is a demonstration. Every abstract operation in the syllabus gets a visible consequence.

2.7 The decoder

$$X[m, k] = \sum_n x[n] w[n - mH] e^{-j2\pi kn/N}$$

The Short-Time Fourier Transform: chop the signal into short frames, transform each one, keep the magnitudes. m is the frame index, k the frequency bin, H the hop between frames.

The decoder already knows every f_r , so it does not search — it looks up:

$$k_r = \arg \min_k |f[k] - f_r|, \quad \hat{I}[r, c] = |X[c, k_r]|$$

$$I_{\text{norm}} = \frac{\hat{I} - \min \hat{I}}{\max \hat{I} - \min \hat{I}}, \quad \hat{B}[r, c] = \begin{cases} 1 & I_{\text{norm}}[r, c] \geq \tau \\ 0 & \text{otherwise} \end{cases}$$

The plan’s nine implementation steps, in order:

1. Load or record the audio.
2. Detect the synchronisation pattern if there is one.
3. Divide the image-bearing region into column frames.
4. Compute the STFT, or one FFT per known frame.
5. Sample the magnitude at each assigned row frequency.
6. Normalise the recovered matrix.

7. Apply the binary threshold, or scale for grayscale.
8. Display and save the recovered image.
9. Score it against the *processed* source image.

Threshold options to expose: a manual slider, an average-energy threshold, and Otsu’s method as the automatic default. Optional clean-up: remove isolated pixels, fill single-pixel gaps, and correct a vertical flip if the row mapping came out reversed.

2.8 Metrics

$$\text{accuracy} = \frac{\text{correct pixels}}{\text{total pixels}}, \quad \text{MSE} = \overline{(\text{source} - \text{recovered})^2}, \quad \text{PSNR} = 10 \log_{10} \frac{\text{MAX}_I^2}{\text{MSE}}$$

For QR mode also report whether the message decoded, as a success rate over repeated trials. For any physical test, repeat every condition several times — room noise and placement vary between runs.

2.9 Recommended starting parameters

PARAMETER	PLAN	THIS REPO	REASON
Sampling rate	44.1 kHz	44.1 kHz ✓	Standard rate, ample bandwidth
Frequency range	800 Hz – 8 kHz	1000 Hz – 8 kHz	Audible, reproducible, avoids rumble
Binary image size	64 × 32 / 64 × 64	64 × 64 ✓	Detail without frequency crowding
Frame duration	20–30 ms	100 ms	See §3.5 — 20 ms is <i>too short</i> for 64 rows
Window	Hann	Hann ✓	Removes boundary clicks and leakage
Output peak	0.8	0.8 ✓	Headroom against clipping
Decoder threshold	Slider + Otsu	—	Experimentation plus a robust default

PART 3

The signal concepts, bit by bit

Fourteen ideas, in the order they hit the signal. Each one says what it is, why this project cannot avoid it, and where it already shows up in the code.

3.1 Sampling and the discrete-time axis

A continuous tone $x(t) = \sin(2\pi ft)$ becomes a list of numbers by evaluating it at regular instants $t = n/f_s$. The sampling rate f_s is how many numbers per second.

```
t = n / f_s , n = 0, 1, 2, ..., N-1
```

This is literally lines 27–28 of `audio_encoder.py`. Two lines that turn continuous mathematics into an array.

Why it matters here: every duration in the project is really a sample count. A 0.1 s frame at 44.1 kHz is 4410 samples, and 64 of those is 282 240 samples — exactly the length of the WAV file in this repository.

3.2 The Nyquist limit

A sampled signal can only represent frequencies below half the sampling rate. Above that, a tone is indistinguishable from a lower one — the information is gone, not merely degraded.

```
f_max < f_s / 2 = 22050 Hz
```

Guarded explicitly at lines 15–16 of `audio_encoder.py`, which raises `ValueError` if you violate it.

Why it matters here: it puts a hard ceiling on the top image row. It also sets the total budget — all R row frequencies must fit between $f_{\min}$ and $f_s/2$ — so *sampling rate limits image height*.

3.3 Sinusoids and superposition

A sinusoid has three properties: amplitude (how loud), frequency (how high) and phase (where in its cycle it starts). Adding two sinusoids of different frequencies gives a signal that still contains both, separably. They do not blend into something new.

THIS IS THE LOAD-BEARING FACT

Superposition is the entire reason a column of 43 simultaneous pixels can be sent as one waveform and pulled apart again at the far end. Frequencies form an *orthogonal basis*; they share the time axis without interfering.

In the code: the inner loop `frame += np.sin(2*np.pi*freq*t)` at line 46 is superposition, one active row at

a time.

3.4 The DFT, the FFT, and bin width

The Discrete Fourier Transform takes N samples of time and returns N numbers indexed by frequency. The FFT is just a fast algorithm for it. The frequencies it reports are not continuous — they sit on a fixed grid.

```
Δf = f_s / N    (bin width)
f[k] = k · f_s / N    (the frequency of bin k)
with f_s = 44100 and N = 4410 → Δf = 10 Hz
```

Bin 800 is exactly 8000 Hz. Bin 100 is exactly 1000 Hz.

Why it matters here: the decoder never searches for a peak. It computes $k_r = \text{round}(f_r N / f_s)$ once and then reads that array position out of every frame. Decoding is an *index lookup*, which is why it is fast and why it barely needs tuning.

3.5 The time–frequency resolution trade-off

Bin width is f_s / N and frame duration is N / f_s . They are reciprocals. A longer frame resolves frequency finely but pins down time coarsely; a shorter frame does the opposite. You cannot have both.

$$\Delta f \cdot T_{\text{frame}} = 1$$

For this project the trade-off has a very concrete consequence. Two adjacent image rows are separated by

$$\text{spacing} = \frac{f_{\text{max}} - f_{\text{min}}}{R - 1} = \frac{7000}{63} = 111.11 \text{ Hz}$$

A Hann-windowed tone occupies a main lobe $4f_s / N = 4/T$ Hz wide. To keep neighbouring rows from merging, that lobe must be narrower than the spacing:

$$\frac{4}{T} < 111.11 \text{ Hz} \implies T > 36 \text{ ms}$$

This is why the plan's suggested 20 ms would smear adjacent rows together at 64 rows, and why the code's 100 ms frame — although it makes the clip 6.4 s instead of 1.28 s — is the safe choice.

THE DESIGN TRIANGLE

Image height (how many rows), **bandwidth** ($f_{\text{max}} - f_{\text{min}}$) and **frame duration** are locked together. Want more rows in the same band? The spacing shrinks, so frames must get longer, so the clip gets slower. Every experiment in the plan is a walk around this triangle.

3.6 Spectral leakage and windows

The DFT assumes the frame repeats forever. If a tone does not complete a whole number of cycles inside the frame, the assumed repetition has a jump at the seam — and a jump contains every frequency. Energy that belongs in one bin *leaks* into all the others.

A **window** fixes this by fading each frame in and out so there is no seam. The cost is a wider main lobe. Different windows trade lobe width against leakage:

WINDOW	MAIN LOBE	FIRST SIDELOBE	CHARACTER
Rectangular (none)	2 bins	−13 dB	Sharpest, leakiest. Audible clicks.
Hann	4 bins	−31 dB	The default. Rolls off fast, 18 dB/octave.
Hamming	4 bins	−43 dB	Lower first sidelobe, slower roll-off.
Blackman	6 bins	−58 dB	Cleanest and widest. Costs resolution.

Measured in this repository: the row frequencies are *not* bin-aligned — only 8 of the 64 land exactly on a bin — so leakage is real. But Hann’s fast roll-off means that eleven bins away (one row’s spacing) it has fallen to nothing. Active cells measure magnitude ≈ 26.9 , silent cells ≈ 0.001 . That is a margin of about **71 dB**, which matches the theory almost exactly.

In the code: `np.hanning(4410)` at line 35, applied at line 50.

3.7 The STFT and the spectrogram

One FFT tells you what frequencies are present in a chunk of sound, but not *when*. The Short-Time Fourier Transform slides a window along the signal and takes an FFT at every position. Stack the magnitude of each result as a column and you have a spectrogram.

A SIMPLIFICATION YOU ARE ALLOWED

The generic STFT has to guess where things start. Yours does not. The encoder writes frames of exactly 4410 samples, back to back, with no overlap — so the decoder can set **N = 4410** and **hop = 4410** and every analysis frame lines up perfectly with one image column. That reduces “run an STFT” to “run one FFT per column”, which is ten lines of NumPy.

Measured: aligned decoding gives 100 % accuracy. Shifting the analysis by half a frame drops it to 95.5 %; by a full frame minus one sample, to 90.2 %. Alignment matters, but it degrades gracefully rather than falling off a cliff.

3.8 LTI systems, impulse response and convolution

A system is **linear** if doubling the input doubles the output and sums pass through unchanged; **time-invariant** if it behaves the same today as tomorrow. Any such system is fully described by one thing: its response to a single impulse, $h[n]$. The output for any input is the convolution

$$y[n] = x[n] * h[n] = \sum_k x[k] h[n - k]$$

An echo is the simplest interesting impulse response — the original plus one delayed copy:

$$h[n] = \delta[n] + \alpha \delta[n - D] \implies y[n] = x[n] + \alpha x[n - D]$$

Why it matters here: because time is the image’s horizontal axis, delaying a copy of the signal *literally shifts a*

ghost of the picture to the right. Convolution stops being an abstract integral and becomes something you can point at on screen.

Measured: with $\alpha = 0.6$, an echo delayed by one tenth of a frame is harmless (100 %); by a full frame it costs a little (95.5 %); by *half* a frame it is worst (83.0 %), because each analysis frame then straddles two different columns.

3.9 Frequency response and filtering

The Fourier transform of $h[n]$ is the frequency response $H(f)$: how much the system multiplies each frequency by. Convolution in time is multiplication in frequency, which is why filters are easiest to think about as a shape drawn over the spectrum.

Here that shape is drawn *over the image's rows*. With $f_r = 8000 - 111.11 r$ you can predict exactly which rows die:

FILTER	FREQUENCIES REMOVED	ROWS REMOVED	MEASURED	ACCURACY
Low-pass @ 4 kHz	$f > 4000$	rows 0–36 (the top 37)	74.9 %	
Band-stop 3–5 kHz	$3000 \leq f \leq 5000$	rows 27–45 (a band)	77.2 %	
Notch @ 5 kHz	$f = 5000$	row 27, alone	≈ 99 %	

Measured on `black1.wav`, whose content occupies only rows 14–52; the rows that went blank were exactly the predicted ones that carried content.

3.10 Aliasing and resampling

Drop the sampling rate below twice a tone's frequency and the tone does not disappear — it **folds**, reappearing at $|f'_s - f|$. It masquerades as a different, lower frequency, and nothing downstream can tell.

```
resample 44100 → 12000 Hz (Nyquist now 6000)
8000 Hz tone folds to 12000 - 8000 = 4000 Hz
```

Row 0 (8000 Hz) would appear at 4000 Hz, which is row 36. The top of the picture is mirrored into its middle.

Why it matters here: this is the sampling theorem made visible. Downsample *with* an anti-alias low-pass first and the top rows are cleanly deleted; downsample *without* one and they come back in the wrong place. Running both and showing the two recovered images side by side is one of the strongest demonstrations in the whole project.

The same idea already appears on the image side: LANCZOS resampling in the preprocessor is a windowed-sinc low-pass applied before decimation — an anti-alias filter in two dimensions.

3.11 Clipping, and why nonlinearity is different

Everything above is linear. Clipping is not: it flattens peaks, and a flattened sinusoid is no longer a sinusoid. It gains harmonics at integer multiples of its own frequency — energy at frequencies that were never transmitted.

a 1000 Hz tone, hard-clipped, gains 3000, 5000, 7000 Hz ...
those land on rows 45, 27 and 9 --- as bright pixels that were never drawn

Measured: clipping at ± 0.4 costs nothing (100 %); ± 0.2 gives 98.1 %; ± 0.1 gives 87.1 %; ± 0.05 gives 76.9 %. It is also the reason the encoder normalises to 0.8 rather than 1.0 — headroom.

3.12 Noise, SNR, and processing gain

Signal-to-noise ratio compares average powers, in decibels:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \frac{P_{\text{signal}}}{P_{\text{noise}}}, \quad \sigma = \sqrt{P_{\text{signal}} / 10^{\text{SNR}_{\text{dB}}/10}}$$

The second form is how you generate noise at a target SNR — you will need it in Phase 4.

Measured robustness of the current encoder, decoded frame by frame:

	+20 DB	+10 DB	0 DB	-5 DB	-10 DB	-15 DB	-20 DB
SNR							
Accuracy	100 %	100 %	100 %	98.9 %	89.2 %	69.8 %	59.4 %

Perfect recovery at 0 dB — noise as loud as the signal — looks surprising until you see why. Noise power spreads across all 2206 frequency bins; a tone’s power sits in one. Concentrating 4410 samples into a single bin buys roughly $10 \log_{10}(N/2) \approx 33$ dB of **processing gain**. Long frames do not just improve frequency resolution, they buy noise immunity too. That is a genuine finding worth putting in the report.

3.13 Normalisation, headroom and crest factor

A column with 43 active pixels sums 43 sinusoids, and the result can peak far above 1.0, which a WAV file cannot hold. Scaling the *whole* waveform by one factor keeps the relative brightness between columns intact — which matters enormously once grayscale mode exists.

Crest factor is peak divided by RMS. Measured on `black1.wav`: peak 0.8, RMS 0.057, so the crest factor is about **14**, or 23 dB. That is high, and the reason is that every sinusoid starts at phase 0, so their waveforms line up near the start of each frame. The plan’s own formula includes a per-row phase φ_r precisely so it can be randomised: scattering the phases lowers the peak, so the normaliser divides by less, so every individual tone ends up louder relative to the noise floor. Free SNR, one line of code.

3.14 Thresholding and scoring

The decoder ends up with continuous magnitudes and must commit to 0 or 1. Three options, in increasing order of robustness:

- **Fixed τ** after min-max normalisation. Fine for clean files — measured here, *any* τ from 0.05 to 0.5 gives 100 %, because the two populations are 71 dB apart.
- **Average-energy threshold** — τ tracks the frame’s own mean. Survives global level changes.
- **Otsu’s method** — picks the τ that minimises within-class variance. No tuning, and it adapts per image. Make this the default and keep the slider for experiments.

For binary images accuracy is the honest metric; PSNR belongs to grayscale mode.

3.15 Cross-reference: where each concept lands in the code

CONCEPT	FILE AND LINES	FORM IT TAKES
Anti-aliased decimation	<code>image_preprocessor.py:32</code>	<code>Image.Resampling.LANCZOS</code>
Quantisation to two levels	<code>image_preprocessor.py:20--22</code>	<code>np.where(image < threshold, 1, 0)</code>
Nyquist limit	<code>audio_encoder.py:15--16</code>	explicit <code>ValueError</code> guard
Row-to-frequency map	<code>audio_encoder.py:18</code>	<code>np.linspace(f_max, f_min, rows)</code>
Discrete time axis	<code>audio_encoder.py:27--28</code>	<code>t = np.arange(N) / f_s</code>
Superposition	<code>audio_encoder.py:43--46</code>	<code>frame += np.sin(...)</code>
Windowing	<code>audio_encoder.py:35, 50</code>	<code>np.hanning</code> , then <code>frame *= window</code>
Framing and concatenation	<code>audio_encoder.py:53</code>	<code>np.concatenate(frames)</code>
Normalisation, headroom	<code>audio_encoder.py:55--61</code>	<code>0.8 * x / max x </code>
Encoder/decoder contract	<code>audio_encoder.py:75--99</code>	<code>metadata.json</code>
Time-domain view	<code>visualize.py:7--12</code>	amplitude against time
Everything else	—	TO DO not written yet

PART 4

Feature inventory

Every feature the plan calls for, scored against what is actually in the repository at commit 2670f22.

FEATURE	STATUS	NOTES
INPUT AND PREPROCESSING		
Image upload (PNG / JPEG / BMP)	DONE	Any format Pillow opens
Grayscale conversion	DONE	<code>.convert("L")</code>
Aspect-preserving resize + pad	DONE	LANCZOS, centred on white
Binary threshold	DONE	Selectable; dark = active
Invert (light = active)	PARTIAL	<code>covert_to_binary1</code> exists but is never called
Crop, brightness, contrast	TO DO	
Drawing canvas	TO DO	Needs the GUI first
Text rendering to bitmap	TO DO	<code>PIL.ImageDraw.text</code> is enough
QR generation with quiet zone	TO DO	qrcode library
Processed-image preview	TO DO	The matrix exists; nothing displays it
ENCODER		
Row-to-frequency assignment	DONE	Descending; top row gets $f_{\max}$
Nyquist validation	DONE	
Column-frame synthesis	DONE	Binary mode
Windowing	PARTIAL	Hann only; the window is not selectable
Global normalisation to 0.8	DONE	
WAV export	DONE	Writes float64 — see §8
Metadata sidecar	DONE	Not in the plan; a genuinely good addition
Adjustable range / rate / frame	PARTIAL	Parameters exist, but <code>frame_duration</code> is overridden at line 24
Grayscale amplitude mode	TO DO	Currently binary only
Per-row phase φ_r	TO DO	All tones start at phase 0
Sync tone / chirp preamble	TO DO	Only needed for the acoustic test
Playback	TO DO	<code>sounddevice</code>
CHANNEL SIMULATOR		
Gaussian noise at target SNR	TO DO	About five lines
Echo via convolution	TO DO	Build $h[n]$, use <code>np.convolve</code>
LPF / HPF / band-stop / notch	TO DO	<code>scipy.signal.butter</code> plus <code>filtfilt</code>
Resampling with/without AA	TO DO	Both variants needed for the comparison
Clipping	TO DO	<code>np.clip</code>

FEATURE	STATUS	NOTES
Independent on/off per effect	TO DO	Checklist requirement
DECODER		
WAV loading	PARTIAL	<code>visualize.py</code> reads one, for plotting only
STFT / per-frame FFT	TO DO	Frames align exactly — this is easy
Target-frequency extraction	TO DO	Bin indices come from <code>metadata.json</code>
Normalise and threshold	TO DO	Manual τ , average-energy, Otsu
Grayscale reconstruction	TO DO	
Synchronisation detection	TO DO	Correlation against the stored chirp
Microphone recording	TO DO	Optional extension
QR reading	TO DO	<code>cv2.QRCodeDetector</code>
ANALYSIS, EVALUATION AND INTERFACE		
Waveform view	DONE	<code>visualize.py</code>
FFT magnitude spectrum	TO DO	
Spectrogram view	TO DO	<code>plt.specgram</code> — the money shot of the demo
Original / processed / recovered / error	TO DO	Four-panel comparison
Accuracy, MSE, PSNR	TO DO	
Accuracy-versus-parameter graphs	TO DO	At least three, per the checklist
Interface, five panels	TO DO	Streamlit is the low-effort route, PySide6 the polished one

WHERE YOU STAND

The encoder — the hard half to get right — is finished and provably correct. The decoder is the smallest remaining piece by a wide margin: with frames aligned and `metadata.json` holding every constant, a working decoder is roughly fifteen lines. The bulk of the remaining work is the channel simulator, the metrics, and the interface that ties them together.

PART 5

The six build phases, and how each is tested

Each phase has a completion target from the plan. What follows adds the part the plan leaves implicit: the specific test, the number that counts as a pass, and what the failure looks like so you recognise it.

5.1 Phase 1 — a reliable processed image matrix (complete)

Target: any input becomes a correct low-resolution binary or grayscale matrix.

How to test it

- Feed a pure-white and a pure-black PNG. Expect `sum == 0` and `sum == R*C`.
- Feed a non-square image. Expect the output to be exactly the target size, with the content undistorted and white bars filling the remainder.
- Feed a known checkerboard. Expect exactly half the cells set.
- Assert the unique values are a subset of $\{0, 1\}$ and the shape is (H, W) .

Failure signatures

Stretched content means the aspect ratio was ignored. An all-ones output means the threshold polarity is inverted. Ragged edges on text mean the resize used nearest-neighbour instead of LANCZOS.

5.2 Phase 2 — the image visibly appears in the spectrogram (complete)

Target: encode, window, normalise, export WAV; waveform and FFT views work.

How to test it

- **Single-row test.** Encode an image with only row r active. FFT one frame; the peak must sit within one bin of `row_frequencies[r]`, and the neighbouring row frequencies must be at least 40 dB down. *Measured here: 71 dB down.*
- **Single-column test.** One active column, the rest blank. The waveform must be silent outside samples $[cN, (c+1)N)$.
- **Length test.** `len(audio) == cols * frame_samples`. *Measured: $64 \times 4410 = 282\,240$ ✓*
- **Peak test.** `max|audio| == 0.8` exactly. *Measured ✓*
- **Orientation test.** Encode an image with only the top row set and confirm the energy is at $f_{\max}$, not $f_{\min}$. This is the vertical-flip bug the checklist warns about.
- **The eyeball test.** `plt.specgram(audio, NFFT=4410, Fs=44100, noverlap=0)` — the picture should be visible.

Failure signatures

Clicks between columns mean the window was not applied. Blurred, merged rows mean the frame is too short for the row spacing (§3.5). A picture that comes out upside down means `linspace(f_min, f_max)` was used instead of the reverse.

5.3 Phase 3 — file-to-file recovery works (next up)

Target: STFT decoder, target-frequency extraction, thresholding, metrics.

How to test it

- **The round trip.** Encode, then decode with no channel. Pass mark: **100 % pixel accuracy**, not 95 %. On a clean WAV there is no reason for a single wrong pixel, and this is already verified achievable — see Part 7.
- **Threshold insensitivity.** Sweep τ from 0.05 to 0.5. On a clean file every value must give the same answer. If accuracy moves with τ , something upstream is wrong.
- **Contract test.** Decode using only `metadata.json` — never by importing constants from the encoder. That is what proves the two halves are genuinely separable.
- **Alignment sweep.** Offset the analysis by 0, $\frac{1}{4}$, $\frac{1}{2}$, $\frac{3}{4}$ of a frame and record accuracy. *Expect 100 / ~ 100 / ~ 95 / ~ 90 %.*
- **Metrics self-check.** Scoring the source against itself must give accuracy 1.0, MSE 0, PSNR infinite. A metric function that cannot pass its own identity test is not trustworthy.

Failure signatures

An accuracy around 65 % that looks like structured noise means you are comparing against the wrong ground truth — this exact trap is documented in Part 7. Every other row wrong means an off-by-one in the bin index. A vertically mirrored result means the frequency array is ascending at one end and descending at the other.

5.4 Phase 4 — controlled channel experiments work

Target: noise, echo, filters, resampling, aliasing and clipping, each independently switchable.

How to test it

- **Identity test first.** With every effect off, the channel must return the input bit-for-bit and accuracy must stay 100 %. Most channel bugs are found here.
- **Noise calibration.** Ask for 10 dB SNR, then measure the SNR you actually produced. They must agree to within a fraction of a decibel.
- **Filter prediction.** Before running the low-pass, *write down* which rows should vanish, from $f_r = 8000 - 111.11 r$. Then run it. The prediction must match the picture — that agreement is the real experimental result, more than the accuracy number.
- **Aliasing pair.** Downsample to 12 kHz with and without the anti-alias filter. Without it, rows above 6 kHz must reappear mirrored around row 36. Side-by-side images are the deliverable.
- **Monotonicity.** Accuracy must fall as any impairment increases. A curve that wanders up and down means the threshold is being re-fitted per condition.

- **Benchmark against Part 7.** Your numbers should land close to the measured values there.

5.5 Phase 5 — a polished software demonstration

Target: GUI integration, image-type comparison, graphs, sensible defaults, error handling.

How to test it

- **Five inputs, one pipeline.** Drawing, text, logo, QR and photograph must all traverse the identical code path. Report accuracy for each; the plan expects at least three categories.
- **Three graphs minimum.** Accuracy against SNR, against frame duration, and against image height. Each needs labelled axes with units — a checklist item.
- **Fresh-eyes test.** Hand the application to someone who has not seen it and ask for one encode and one decode without help. Wherever they hesitate is your interface bug.
- **Hostile inputs.** A 1×1 image, an all-white image, a 4000×3000 photograph, a corrupt WAV, and a WAV whose length is not a multiple of the frame size. None may crash.
- **Responsiveness.** Nothing may block the UI thread long enough to freeze it; run long experiments off the interface thread.

5.6 Phase 6 — speaker to microphone (optional extension)

Target: physical playback and recording, synchronisation, final deliverables.

How to test it

- **Loopback first.** Route the output straight back to the input in software. If that fails, the room is not your problem.
- **Synchronisation accuracy.** Play with a deliberate 2 s silent lead. The detected start must land within a few samples of the truth; feed it a known offset and check the number it reports.
- **Repeat everything.** Five runs per condition, reporting mean and spread. A single successful acoustic run is an anecdote.
- **Frequency-response check.** Encode an all-ones image and record it. The measured magnitude per row is the speaker-microphone frequency response. Expect the top rows to sag — that measurement is itself a result worth plotting.
- **Document the failures.** The checklist explicitly asks for known limitations. An honest account of what did not work reads better than a claim that everything did.

PART 6

The existing code, line by line

Three files, 188 lines. For each block: what the Python does, and — more importantly — what state the signal is in before and after it.

6.1 encoder/image_preprocessor.py

Turns an arbitrary image file into the 64×64 matrix of ones and zeros that the encoder will sing. Nothing here is audio yet; this is the whole “picture” half of the pipeline.

```

1  from PIL import Image
2  import numpy as np
3
4  def preprocess_image(image_path):
5      image = Image.open(image_path).convert("L")
6      img_format = image.mode
7      height = image.height
8      width = image.width
9
10     # print(img_format)
11     # print(height)
12     # print(width)
13
14     return image

```

Lines 1–2 — imports

Pillow for file decoding and geometry, NumPy for the array maths. No SciPy here, so this file runs on a bare install.

Lines 4–5 — load and flatten to grayscale

`convert("L")` collapses three colour channels into one luminance channel using the ITU-R 601-2 weighting $L = 0.299R + 0.587G + 0.114B$ — green counts most because the eye is most sensitive to it.

state 1 → 2 encoded image file → 2-D intensity field, uint8 0 to 255, full resolution

Lines 6–12 — inspection leftovers

Three locals are assigned and never used, and three prints are commented out. Debug scaffolding from when the file was written; harmless.

Line 14

Returns a *Pillow object*, not a NumPy array — deliberate, because the next stage needs Pillow’s resampling.

```

16  def covert_to_binary1(image, threshold=128):

```

```

17     binary_image = np.where(image >= threshold, 1, 0)
18     return binary_image
19
20 def covert_to_binary2(image, threshold=128):
21     binary_image = np.where(image < threshold, 1, 0)
22     return binary_image

```

Lines 16–22 — two opposite polarity choices

Version 1 makes *bright* pixels active; version 2 makes *dark* pixels active. Only version 2 is wired up.

That choice has an audible meaning: **dark ink becomes a tone, white paper becomes silence**. Draw black letters on white and the letters are what you hear. It also pairs correctly with the white padding added on line 36 — margins map to 0, which is silence, so padding costs nothing in the audio.

Version 1 is exactly the “invert” control the plan asks for; it just needs exposing as a flag. Both names carry the same typo, `covert` for `convert` — consistently, so nothing breaks.

```
state 3 → 4   uint8 0--255   →   int 0/1, "is there a tone here"
```

```

24 def resize_image(image, target_width= 16, target_height = 16):
25     width, height = image.size
26
27     scale = min(target_width / width, target_height / height)
28
29     new_width = max(1, round(width * scale))
30     new_height = max(1, round(height * scale))
31
32     resized_image = image.resize((new_width, new_height), Image.Resampling.LANCZOS)
33
34     # Lancz05 resampling algorithm , grayscale desampling er jonno good
35
36     background = Image.new("L", (target_width, target_height), 255)
37     x_padding = (target_width - new_width) // 2
38     y_padding = (target_height - new_height) // 2
39     background.paste(resized_image, (x_padding, y_padding))
40
41     return background

```

Line 27 — the fit-inside scale factor

Taking the *minimum* of the two ratios guarantees both dimensions land inside the target. Using the maximum would overflow; using them independently would stretch. This one `min` is what satisfies the plan’s “pad, never stretch” rule.

Lines 29–30

`max(1, ...)` guards the degenerate case where an extreme aspect ratio would round a dimension to zero.

Line 32 — the most signal-theoretic line in the file

LANCZOS resampling convolves with a windowed `sinc` kernel before decimating: a two-dimensional anti-alias low-pass. Without it, shrinking a 3000-pixel-wide photo to 64 columns would fold fine texture into false low-frequency patterns — *image-domain aliasing*, the exact phenomenon §3.10 studies in the audio domain. The

comment beside it, in mixed Bangla and English, says the same thing: Lanczos is good for grayscale downsampling.

state 2 → 3 full-resolution grayscale → band-limited, decimated grayscale, aspect preserved

Lines 36–39 — centre onto a white canvas

A new L-mode image of the exact target size filled with 255, with the resized picture pasted at the centre. The floor division splits the leftover space evenly.

The fill value matters: 255 is white, white is above the threshold, so under `covert_to_binary2` the margins become 0 — silence. Had it been filled with 0 (black), every padded row would blare all 64 tones for the whole clip.

```

43 def process(image_path, target_width=16, target_height=16, threshold=128):
44
45     grayscale_image = preprocess_image(image_path)
46
47     processed_image = resize_image(
48         grayscale_image, target_width, target_height)
49
50     image_array = np.array(processed_image)
51
52     # print("After resize:")
53     # print(image_array.shape)
54
55     binary_image = covert_to_binary2(image_array, threshold=threshold)
56
57     # print("Binary shape:")
58     # print(binary_image.shape)
59     # print("Unique values:")
60     # print(np.unique(binary_image))
61     # print(binary_image)
62     # np.savetxt("binary_image.txt", binary_image, fmt="%d")
63
64     return binary_image
65
66
67 if __name__ == "__main__":
68     process("images/cat2.jpg")

```

Blank lines 44–46 and a few comment lines have been closed up here so the listing fits; the line numbers of every statement match the file exactly.

Lines 43–52 — the pipeline in order

Load and grayscale, resize and pad, then hand to NumPy. `np.array` on an L-mode Pillow image gives shape (height, width) — *rows first*. That ordering is what makes `binary_image[:, idx]` in the encoder a column, and it is the reason the row/column convention holds all the way through.

Line 57

The dark-is-active thresholder. Output dtype is the platform integer type, values strictly 0 and 1.

returns state 4 the binary pixel matrix, shape (64, 64) --- the encoder's only input

Line 66 — worth knowing

This commented-out `np.savetxt` is what once wrote `binary_image.txt` in the project root. Because it is disabled, that file is a *stale snapshot of a different image* from an earlier commit. It matches the current `black1.png` only 65 % of the time. Do not use it as ground truth — regenerate by calling `process()`. Part 7 records how this cost real debugging time.

Lines 71–72 — currently broken

This references `images/cat2.jpg`, which was deleted in commit `2670f22`. Running the file directly raises `FileNotFoundError`. The `process()` function itself is fine — only this self-test line is stale.

6.2 encoder/audio_encoder.py

Where pixels become sound. This is the heart of the project.

```

1  import numpy as np
2  from scipy.io.wavfile import write
3  from image_preprocessor import process
4  import json
5
6  def encode(image_path, target_width=16, target_height=16, output_file='output.wav',
7            sampling_rate=44100, f_min=1000, f_max=8000,
8            frame_duration=0.1, threshold=128):
9
10     binary_image = process(image_path, target_width=target_width,
11                           target_height=target_height, threshold=threshold)
12     rows = binary_image.shape[0]
13     cols = binary_image.shape[1]
14
15     # f_max should be less than sampling_rate / 2 - Nyquist frequency
16     if f_max >= sampling_rate / 2:
17         raise ValueError("f_max should be less than sampling_rate/2")
18
19     row_frequencies = np.linspace(f_max, f_min, rows)
20
21     # print(row_frequencies[0])
22     # print(row_frequencies[-1])
23     # print(row_frequencies[1]- row_frequencies[0])
24
25     frame_duration = 0.1
26     frame_samples = int(sampling_rate*frame_duration)
27
28     n = np.arange(frame_samples)
29     t = n / sampling_rate

```

Lines 1–4 — imports

NumPy for synthesis, `scipy.io.wavfile.write` for export, the preprocessor for input, `json` for the sidecar. The bare `from image_preprocessor import ...` means the `encoder/` directory must be on the import path — which happens automatically when you run `python encoder/audio_encoder.py` from the project root.

Lines 6–8 — every parameter in one signature

Image size, output path, sampling rate, frequency band, frame duration, threshold. This is already the parameter set the plan's Encoder panel needs to expose; the GUI can bind straight onto it.

Lines 10–12

Runs the whole preprocessing chain, then reads the shape. `rows` becomes the number of distinct frequencies, `cols` the number of time frames — the two axes of the spectrogram are established right here.

holds state 4 binary matrix (rows x cols)

Lines 14–16 — the Nyquist guard

Refuses to encode if the top frequency would exceed half the sampling rate, because above that limit the tone cannot be represented at all — it would come back as a different, lower frequency and paint the wrong row. Failing loudly here beats debugging a mirrored image later. With the defaults, $8000 < 22050$, comfortably legal.

Line 18 — the row-to-frequency map; note the argument order

`linspace(f_max, f_min, rows)` counts *down*: row 0 gets 8000 Hz, row 63 gets 1000 Hz. The plan's written formula ascends; this deliberate reversal is what satisfies the plan's stated requirement that the top image row take the highest frequency, so the spectrogram is not upside down. It is also recorded in the metadata as "top_high to bottom_low".

Spacing works out to $7000/63 = 111.11$ Hz per row — the number that governs every resolution decision in §3.5.

Line 24 — flag this one

`frame_duration` is a function parameter, and this line overwrites it with a hard-coded 0.1 before it is used. Calling `encode(..., frame_duration=0.02)` silently produces 0.1 s frames anyway.

The value itself is a good choice — §3.5 shows 100 ms is safe where 20 ms would not be. But the frame-duration sweep is one of the plan's headline experiments, and it cannot run while this line stands. Deleting it is a one-line fix, and it belongs to whoever owns this file.

Line 25

`int(44100 * 0.1) = 4410 samples per column`. Continuous duration becomes a sample count — the moment the design becomes discrete.

Lines 27–28 — the sampling equation, in two lines

n is the sample index 0 to 4409; $t = n/f_s$ is the instant each sample represents. Every sinusoid below is evaluated on this same shared axis, which is what allows them to be added coherently.

creates the discrete time base `t = [0, 22.7 us, 45.4 us, ... , 99.98 ms]`

```

30     # 1 column = 1 frame = frame_samples
31     # binary image er prottek row er jonno column er jekhane jekhane 1 ogula jog hobe
32     # like  $x[n] = \text{Asin}(2 * \pi * f_1 * t) + \text{Asin}(2 * \pi * f_2 * t) + \dots$ 
33
34     frames = []
35     window = np.hanning(frame_samples)
36
37     for idx in range(cols):
38
39         col = binary_image[:, idx]
40         one_rows = np.where(col == 1)[0]
41         frame = np.zeros_like(t, dtype=np.float64)
42
43         for row in one_rows:
44
45             freq = row_frequencies[row]
46             frame += np.sin(2 * np.pi * freq * t)
47
48     # sharp transitions = Spectral Leakage

```

```

49     # A Hann window smoothly changes frame amplitude
50     frame *= window
51     frames.append(frame)
52
53     final_audio = np.concatenate(frames)

```

Lines 30–32 — the author’s own note

In mixed Bangla and English: *one column = one frame*, and for each row of the binary image, wherever there is a 1, those get summed — $x[n] = A \sin(2\pi f_1 t) + A \sin(2\pi f_2 t) + \dots$. That is the encoder equation stated plainly.

Line 35 — the window, computed once

`np.hanning(4410)` is a raised cosine that starts at 0, peaks at 1 in the middle and returns to 0. Computing it outside the loop and reusing it is both faster and a guarantee that every frame is treated identically — a checklist requirement.

Lines 37–39 — column by column, left to right

`binary_image[:, idx]` takes all rows at one column: a vertical slice of the picture, which becomes one instant of the sound.

Line 40

`np.where(col == 1)[0]` returns the indices of the active pixels. This is the sparse optimisation: instead of adding 64 tones scaled by 0 or 1, it only touches the rows that are on. For a typical column here that is 8 to 43 of the 64.

Line 41

A silent frame to accumulate into — explicitly `float64`, which is why the exported WAV ends up in 64-bit float format (see Part 8).

Lines 43–46 — superposition, one row at a time

Look up the row’s frequency, generate a full-length sine over the shared time axis, add it in. After the loop the frame contains every active row’s tone simultaneously, each perfectly separable by frequency.

Two details worth naming. Amplitude is implicitly 1.0 for every tone — that is binary mode; grayscale mode is the same line with `col[row]/255 *` in front of it. And every sine starts at phase 0, where the plan’s formula allows a per-row φ_r ; see §3.13 for what that costs.

state 4 → 5 a column of pixels → 4410 float64 samples containing that column's chord

Lines 48–50 — apply the window

The comment gives the reason: a frame that starts and stops abruptly has step discontinuities at its edges, and a step contains every frequency — energy smears across the whole spectrum, blurring the picture and adding an audible click at every column boundary. The Hann taper fades each frame in and out so the seams vanish.

Cost: the main lobe widens from 2 bins to 4, which is what makes the 36 ms minimum frame duration in §3.5 bind. Benefit, measured: 71 dB of separation between an active row and a silent one.

state 5 → 6 raw chord → tapered frame, zero at both ends

Line 53 — frames become a signal

64 frames of 4410 samples, laid end to end with no overlap, give 282 240 samples = 6.4 seconds. The lack of overlap is a gift to the decoder: analysis frame c is exactly samples $[4410c, 4410(c+1))$, with no hop arithmetic required.

state 6 → 7 64 separate frames → one continuous waveform

```

55     peak = np.max(np.abs(final_audio))
56     # print(peak)
57
58     # Normalization
59
60     if peak > 0:
61         final_audio = 0.8 * final_audio / peak
62
63     # print(np.max(np.abs(final_audio)))
64
65     write(output_file, sampling_rate, final_audio)
66     # print(binary_image.shape)
67     # print(np.unique(binary_image))
68
69
70     create_metadata_json(sampling_rate, rows, cols,
71                         row_frequencies, f_min, f_max, frame_duration,
72                         frame_samples, threshold)

```

Lines 55 and 60–61 — global normalisation with headroom

One peak taken across the entire waveform, one scale factor applied to the entire waveform. Two consequences.

First, *global* rather than *per-frame* is the right call: it preserves the relative loudness between columns, which is exactly the information grayscale mode will carry. Per-frame normalisation would make every column equally loud and destroy that.

Second, *0.8* rather than *1.0*. The remaining 20 % is deliberate headroom, so that later stages — a channel effect, a resampler’s ringing, a soundcard’s own gain — do not push the signal into clipping, which as §3.11 shows would fabricate rows that were never drawn.

The `if peak > 0` guard handles a fully blank image without dividing by zero. Measured result: peak exactly 0.8, RMS 0.057, crest factor about 14.

state 7 → 8 arbitrary amplitude → peak-normalised to 0.8, safe to write

Line 65 — export

`scipy.io.wavfile.write` picks the WAV sample format from the array’s dtype. Because `final_audio` is `float64`, the file is written as 64-bit IEEE float, format tag 3 — confirmed in the header of `black1.wav`. Nothing is lost, but

it is an unusual format; see Part 8.

state 8 → 9 NumPy array → playable mono WAV, 44.1 kHz

Lines 70–72

Writes the sidcar. Note it passes `frame_duration` — the value from line 24, not the caller’s — so the metadata is at least self-consistent with what was actually generated.

```

75 def create_metadata_json(sampling_rate, rows, cols, row_frequencies,
76                          f_min, f_max, frame_duration,
77                          frame_samples, threshold):
78     metadata = {
79         "sample_rate": sampling_rate,
80         "rows": rows,
81         "columns": cols,
82         "row_frequencies": row_frequencies.tolist(),
83
84         "f_min": f_min,
85         "f_max": f_max,
86
87         "frame_duration": frame_duration,
88         "frame_samples": frame_samples,
89
90         "encoding_mode": "binary",
91         "window": "hann",
92
93         "active_pixel_value": 1,
94         "threshold": threshold,
95         "frequency_mapping": "top_high to bottom_low"
96     }
97
98     with open("metadata.json", "w") as file:
99         json.dump(metadata, file, indent=4)
100
101
102 if __name__ == "__main__":
103     encode("images/black1.png", target_width=64, target_height=64,
104           output_file='output.wav',)

```

Lines 75–96 — the key to the cipher

This is the best idea in the codebase. The plan never asks for it, but the implementation checklist demands that “the decoder uses the same frequency map and frame duration as the encoder” — and this is how you guarantee that without the two halves importing each other.

It stores all 64 row frequencies explicitly rather than storing $f_{\min}$ and $f_{\max}$ and asking the decoder to re-derive them, which removes any chance of the two sides disagreeing about rounding or direction. It records the orientation in words. It even records the encoding mode and the window, so a future grayscale or Blackman mode will not silently break an old decoder.

Build the decoder to read only this file. Never import a constant from the encoder.

Lines 98–99

Written to the current working directory, with a fixed name. So every encode overwrites the previous key, and a WAV and its metadata can drift apart if you generate several files. Naming it after the output — `output.wav` to

output.json — would pair them permanently. Worth raising, not urgent.

Lines 102–103 — the self-test

Encode images/black1.png at 64×64 to output.wav. The trailing comma in the call is legal Python and harmless.

Run it as `python encoder/audio_encoder.py` **from the project root** — that is the only working directory where both the bare import and the relative image path resolve. This is almost certainly why `tempCodeRunner-File.py` exists: VS Code’s Code Runner changes directory into the file’s folder, which breaks the image path.

6.3 encoder/__pycache__/visualize.py

Twelve lines that close the loop back to your eyes — the first step of the analysis half.

```

1  from scipy.io.wavfile import read
2  import matplotlib.pyplot as plt
3  import numpy as np
4
5  fs, audio = read("black1.wav")
6
7  time = np.arange(len(audio)) / fs
8
9  plt.plot(time, audio)
10 plt.xlabel("Time (s)")
11 plt.ylabel("Amplitude")
12 plt.show()
```

Line 5

`read` returns the sample rate and the samples, with the dtype matching the file — here `float64`, already within ± 0.8 , so no scaling is needed. The path is relative, so this must run from the project root.

state 9 → 8 WAV file → NumPy array, back in memory

Line 7

Rebuilds the time axis — the inverse of line 28 in the encoder. The same equation $t = n/f_s$, run in the opposite direction, now spanning the full 6.4 s rather than one frame.

Lines 9–12 — the time-domain view, and its lesson

Plotted this way the signal looks like a dense band of noise with 64 gentle bulges: the Hann envelopes of the columns. The picture is completely invisible. That is not a failure, it is the point. *The information is in the frequency content, and the time domain cannot show frequency content.*

This plot is the “before” that makes the spectrogram’s “after” land. Keep both in the presentation, side by side. The natural next line — `plt.specgram(audio, NFFT=4410, Fs=fs, noverlap=0)` — is where the image appears.

ONE PRACTICAL WARNING

This file lives inside `encoder/__pycache__`. That directory is Python's bytecode cache; any tool that clears caches, and most `.gitignore` templates, will delete it. It is real source code sitting in a disposable folder. Moving it to `analysis/visualize.py` costs nothing and protects it.

6.4 The two supporting files

metadata.json — 80 lines

The generated key for `black1.wav`: 44100 Hz, 64×64 , all 64 row frequencies from 8000.0 down to 1000.0, 4410 samples per frame, binary mode, Hann window, threshold 128, orientation `top_high` to `bottom_low`. Everything the decoder needs, and nothing it has to guess.

encoder/tempCodeRunnerFile.py — 1 line

A single stray line, `write(output_file, sampling_rate, final_audio)`. This is a VS Code “Code Runner” scratch file, created automatically when you run a selection rather than a whole file. It is not part of the project and can be deleted.

PART 7

Measurements taken from the real output

Every number in this guide came from running against the committed `black1.wav`, with ground truth regenerated by calling `process("images/black1.png", 64, 64, 128)`. No code in the repository was modified.

THE HEADLINE

Decoding `black1.wav` with a plain per-frame FFT — 4410-point, aligned to the encoder’s frames, reading the magnitude at each of the 64 known row frequencies — recovers the source matrix with **100 % pixel accuracy**. Active cells measure ≈ 26.9 , silent cells ≈ 0.001 . Every threshold from 0.05 to 0.5 gives the same perfect answer.

What this means for you: the encoder is not “probably right”, it is verified. Phase 3 is not a research problem — the signal is clean and waiting. Fifteen lines will finish the round trip.

Signal properties

Samples	282 240 = 64 × 4410 ✓
Duration	6.40 s
Peak	0.800 ✓
RMS	0.0573
Crest factor	13.97 (22.9 dB)
WAV format	IEEE float, 64-bit, mono
Row spacing	111.11 Hz
FFT bin width	10.0 Hz
Hann main lobe	40 Hz (4 bins)
Bin-aligned rows	8 of 64
Active/silent margin	≈ 71 dB
Active pixels in source	1701 of 4096

Accuracy under impairment

CONDITION	ACCURACY
Clean, aligned	100.0 %
Analysis offset $\frac{1}{4}$ frame	100.0 %
Analysis offset $\frac{1}{2}$ frame	95.5 %
Analysis offset ~ 1 frame	90.2 %
Noise @ 0 dB SNR	100.0 %
Noise @ -10 dB SNR	89.2 %
Noise @ -20 dB SNR	59.4 %
Echo α 0.6, delay 0.1 fr	100.0 %
Echo α 0.6, delay 0.5 fr	83.0 %
Echo α 0.6, delay 1.0 fr	95.5 %
Low-pass @ 4 kHz	74.9 %
Band-stop 3–5 kHz	77.2 %
Clip at ± 0.2	98.1 %
Clip at ± 0.05	76.9 %

A TRAP THAT COST REAL TIME

The first decode attempt scored 65 % and produced a plausible-looking but wrong image. The decoder was fine. The *ground truth* was wrong: `binary_image.txt` in the project root was written by an earlier commit from a different photograph, and the `np.savetxt` that made it is now commented out at line 66. It agrees with the current `black1.png` only 65.3 % of the time — which is exactly the accuracy that appeared.

Regenerating the truth by calling `process()` took the score straight to 100 %. When Phase 3 gives you a number in the sixties, suspect your reference before your maths.

PART 8

Open issues to raise with your teammate

Observations only — nothing here has been changed. These are the encoder author's calls to make.

OBSERVATION	PRIORITY	DETAIL
frame_duration overridden	TO DO	audio_encoder.py:24 reassigns the parameter to 0.1, so the argument never takes effect. The frame-duration sweep is a required experiment and cannot run until this line goes. The value is a good default — make it the signature default instead.
Stale binary_image.txt	TO DO	Left over from an earlier image; 65 % agreement with the current source. Either regenerate it or delete it, so nobody trusts it. See Part 7.
Broken self-test path	PARTIAL	image_preprocessor.py:72 points at images/cat2.jpg, deleted in commit 2670f22. Running the file directly raises FileNotFoundError.
visualize.py inside __py-cache__	PARTIAL	Real source in a disposable directory. One cache clean and it is gone. Move it out.
WAV written as float64	PARTIAL	Format tag 3, 64-bit, because the array is float64. Perfectly valid and lossless, but double the file size and not universally readable — Python's own wave module refuses it. final_audio.astype(np.float32) before writing, or int16 for maximum portability, would be safer for the acoustic test.
All tones start at phase 0	PARTIAL	Crest factor measures 14. Randomising φ_r — which the plan's own formula includes — flattens the peak, so the 0.8 normalisation divides by less and every tone gains level against the noise floor. Cheap SNR.
Grayscale mode absent	PARTIAL	Binary only. The change is one line — scale by <code>col[row]/255</code> at audio_encoder.py:46 — plus recording "encoding_mode": "grayscale". Needed for the PSNR experiments.
Window not selectable	PARTIAL	Hann is hard-coded. The window comparison (rectangular, Hann, Hamming, Blackman) is a listed experiment; the metadata already has a "window" field waiting for it.
Frequencies not bin-aligned	PARTIAL	Only 8 of 64 land on an exact FFT bin, so leakage exists in principle. Measured, it is 71 dB down and harmless — but the checklist mentions bin alignment, and snapping each f_r to the nearest bin costs one line if you want to close the point.
metadata.json naming	PARTIAL	Fixed filename in the working directory, so each encode overwrites the last and a WAV can drift from its key. Pair it with the output name instead.
Unused covert_to_binary1	PARTIAL	Dead today, but it is precisely the “invert black and white” control the plan asks for. Expose it as a flag rather than deleting it.
Local environment	PARTIAL	This machine's Python has NumPy, Pillow and Matplotlib but not SciPy . The preprocessor runs; the encoder does not until <code>pip install scipy</code> .

SUGGESTED NEXT MOVE

Phase 3 is yours in the two-person split, and the ground under it is solid. Build `decoder/stft_decoder.py` so that it reads *only* `metadata.json` and a WAV path, returns the magnitude matrix, and knows nothing about the encoder. Then `decoder/image_reconstructor.py` for `normalise-and-threshold`, and `analysis/metrics.py` for accuracy, MSE and PSNR. Aim the first commit at exactly one number: 100% on `black1.wav`. Part 7 says it is there.

